# Juan Manuel **Young Hoyos**

Software Engineer, Full-Stack — TypeScript · Systems, Real-Time & WebAssembly

*Medellín, Colombia · Open to relocating to Amsterdam (on-site) · English B2+*

+57 1234567890 | juanmanuel12.13jmyh81@gmail.com | jmyounghoyos.com | Youngermaster | juan-manuel-young-hoyos


**Software Engineering — Hiring Team**

*August 12, 2026*

Monumental

**Application for Software Engineer, Full-Stack**

Dear Monumental Team,

I am applying for the **Full-Stack Software Engineer** role at Monumental. Software that touches the physical world is the reason I am writing: I would rather debug why a robot behaves differently than its simulation than ship another SaaS screen. Building in Amsterdam, on-site, around a robot testing floor, is part of the appeal rather than a cost.

Your stack is **TypeScript and Rust, much of it compiled to WebAssembly** so the UI shares control code with the robots. Both halves of that are already how I spend my time. I run **Project Olivaw** (github.com/Project-Olivaw), robotics libraries in pure **Rust**: **olivaw-slam**, a 2D lidar **SLAM** implementation with correlative scan matching, keyframing, a log-odds occupancy grid and pose estimation — built deliberately without ROS2 or C++ dependencies — and **olivaw-lidar**, a driver that talks to **RPLIDAR hardware over serial** with no C++ SDK or FFI and cross-compiles to **aarch64** boards, streaming live scans into **Rerun**. That last part is your first bullet: telemetry that lets you see what a robot is actually doing in real time. On the WebAssembly side, I refactored a legacy **C++ WASM** application (+**45%** code quality), so I understand what that shared-stack boundary costs and buys.

The problems you describe are ones I have versions of. **Telemetry so you can see what every unit is doing:** as Technical Lead at Okorum I stood up logs, traces and metrics across a distributed system and engineered **fault tolerance and resumability** into its queues, so in-flight work survives an unreliable external dependency that goes down without warning — the software equivalent of a flaky link on a site. **Choosing the right abstractions:** I picked the core architecture for that platform and shipped it ahead of a hard government deadline. **Real-time and protocols:** I drove an **MQTT** architecture at ArcDesign with device **simulations in Python** before integrating the logic for near real-time behaviour, and I have built event-driven systems on Kafka and RabbitMQ.

On **LLM agents**: I run several in parallel — **Claude Code** for building, a separate pass for code review, and others for investigation and documentation — and I am usually the person colleagues ask when an agent does something inexplicable. I treat their output as a draft to be validated, not a result.

On foundations: I hold a B.S. in Computer Science and Engineering, I built a geohash-and-hashmap spatial algorithm for scalable geolocation, and I have done 3D engine work in Godot and Unity, so transforms and linear algebra are familiar ground. I am comfortable on Linux, SSH-ing into a machine to work out why something is not behaving.

Thank you for your consideration — the system below is the clearest answer I can give on ownership.

## Most Complex System Owned End-to-End — Rediat

**The problem.** Colombia's **Resolución 1888 de 2025** required every healthcare provider in the country to submit a **Resumen Digital de Atención en Salud (RDA)** — a standardised digital summary of each clinical encounter — as **HL7 FHIR R4**, by **15 April 2026**. The deadline was statutory. Providers had legacy systems, no FHIR experience, and a national platform that was slow, intermittently unavailable and thinly documented. We started in December 2025 with almost nothing.

**What I owned.** I chose the architecture and built it from the ground up: a **Turborepo monorepo of TypeScript/NestJS microservices** streaming send → validate → convert → generate stages over **message queues** into standards-compliant FHIR resources. Because the upstream government API failed unpredictably, I engineered **fault tolerance and resumability** into the queues so in-flight submissions recover instead of being lost, added a **Redis** caching layer so the same clinical codes are not re-validated on every request, and modelled **MongoDB** persistence for **high-throughput** multi-provider load. I built **Python/FastAPI** services for provider analytics with long-running aggregation offloaded to background workers, the **React** front end, **CI/CD on AWS**, and the **observability** (Grafana, Loki) that showed us which stage was failing.

I also owned the customer-facing half, which mattered as much as the code. I wrote the API documentation and ran long technical sessions with hospitals' health-IT leads so they could integrate and become compliant. We ended up with clearer documentation than the government's own, and I pushed the product to explain itself: actionable error suggestions, **automatic correction of the most common submission errors**, and messages in Spanish rather than English — small things that removed a lot of friction for clinical staff.

**Outcome.** We delivered **ahead of the mandate deadline**. Roughly **10% of all production RDAs for national health events** now flow through Rediat. We found product-market fit within a few months of starting, and the team has been growing since.

Sincerely,

**Juan Manuel Young Hoyos**